

データ分析の実際 2

～ 文章データ分析 (自然言語処理) ～

文章の構造

- ・ **文字** ... 表音文字, 表意文字
↓
- ・ **形態素** ... 機能のある最小単位 (≡ **単語**)
(自立語, 付属語, 接頭辞, 接尾辞など)
↓
- ・ **文** ... 叙述の一単位, **統語構造**を持った形態素の並び
↓
- ・ **文章** ... 一連の文からなる叙述, **文脈**を持つ

※ 意味に踏み込んだ処理では, 統語構造や文脈も考慮する必要がある

1

2

統語規則・統語構造

- ・ **統語規則**
 - 文や発話における形態素 (単語) の並びを規定する規則
 - いわゆる「文法」
- ・ **統語構造**
 - 統語規則を反映した, 文の単語 (形態素) 間の関係の構造

文脈

- ・ 文章を構成する一連の文の背後にある意味的な連関, 共通知識など

3

形態素解析

- ・ 文章を**形態素** (単語) に分けていく処理
 - 分かち書きをする言語 (例えば英語) の場合:
空白や記号を区切りとみなせる
 - 分かち書きをしない言語 (例えば日本語) の場合:
区切りが明示されないなので, 辞書的な情報が必要になる
- ・ 各形態素に, 品詞・(語尾)変化・原形などの情報も付与する
 - 適切な解析には辞書的な情報が必要 (意味は必ずしも必要ない)

4

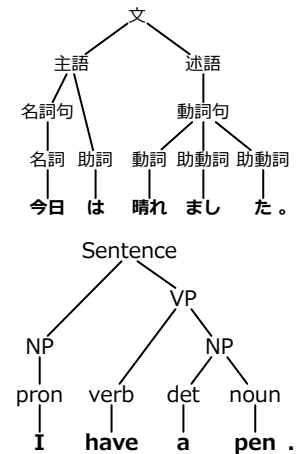
N-gram 解析

- N-gram : 文章中出现する, 連続したN文字のパターン
- N-gram 解析
 - 文章中出现する N-gram や, その出現頻度, 分布等を解析する
 - 形態素や統語構造に関係なく, 文字の並びのパターンにのみ注目するので, 処理の負担は小さく, 言語にも依存しない。

5

統語解析 (構文解析)

- 文章全体の漠然とした意味内容については, 単語単位で意味を見ていくことでもある程度把握できるが, 文章の意味を正確にとらえるためには, 統語構造をとらえることも必要
- 文の統語構造の解析を統語解析 (構文解析)という
 - 統語規則に従って, 構文木 (右図) を構成するなどの手法が一般的



6

文章データ分析 (自然言語処理) の応用例

- 検索 : Webページなどのテキストデータから文字列を探す
 - 従来は検索文字列をそのまま用いていたが, 最近は形態素解析を使ったものが多い。例えば「神戸女学院大学」の検索で「神戸女学院」と「大学」の両方を含むものがヒットするなど
- 校正 : より良い文章になるように, 書き換えを提案する
- トピックモデル : その文章が何に関するものか判別する
 - 例えば, スポーツ関係か, 経済関係か, 芸能関係かなど
- センチメント解析 : 情緒的に肯定的か否定的かを評価する

7

文章データ分析 (自然言語処理) の応用例

- パラフレーズ解析 : 2つの表現が同じ内容かどうかを判別する
 - または, ある表現と同じ内容の別の表現を生成することなど
- 要約 : 長い文章を, 内容を損なわずに短くする
- 対話 : 会話したり, 質問に答えたりする
- 文章生成 : 俳句や小説などを自動的に作らせる

8

文章データ分析 (自然言語処理) の応用例

- 迷惑メールの判定 :

- 迷惑メールとそうでない通常のメールの多数の事例から学習し, 新規に届いたメールが迷惑メールか否かを判定する
- ナイーブベイズなどの手法が知られている

ナイーブベイズによる迷惑メール判定

以下のような事象を考える

- S : メールが迷惑メールである
- A_i : メールが語 w_i を含む ($i = 1 \dots k$)

この時, 語 $w_1, \dots, w_k$ の全てを含むメールが迷惑メールである確率は

$$P(S|A_1 \cap \dots \cap A_k)$$

で表される

ナイーブベイズによる迷惑メール判定

ベイズの定理 (下の等式) を用いる

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

$$\left(\begin{array}{l} \text{これは,} \\ P(A|B)P(B) = P(A \cap B) = P(B|A)P(A) \\ \text{からただちに導ける} \end{array} \right)$$

ナイーブベイズによる迷惑メール判定

この確率 $P(S|A_1 \cap \dots \cap A_k)$ は, ベイズの定理より

$$P(S|A_1 \cap \dots \cap A_k) = \frac{P(A_1 \cap \dots \cap A_k|S)P(S)}{P(A_1 \cap \dots \cap A_k)}$$

と表せるが, 右辺の $P(A_1 \cap \dots \cap A_k|S)$ の計算は一般に困難なので, S の条件下で各 A_i が独立であることを仮定すると,

以下の式が得られる.

$$P(A_1 \cap \dots \cap A_k|S) = P(A_1|S) \dots P(A_k|S)$$

ナイーブベイズによる迷惑メール判定

前スライドの議論から,

$$P(S|A_1 \cap \dots \cap A_k) = \frac{P(A_1|S) \dots P(A_k|S)P(S)}{P(A_1 \cap \dots \cap A_k)}$$

という式が得られるが, 先の各 A_i が独立であるという仮定は必ずしも正しくないので, この $P(S|A_1 \cap \dots \cap A_k)$ と, 同様に計算した $P(\bar{S}|A_1 \cap \dots \cap A_k)$ の和は1になるとは限らない

そこで, 計算した $P(S|A_1 \cap \dots \cap A_k)$ と $P(\bar{S}|A_1 \cap \dots \cap A_k)$ の比較で, 迷惑メールか否かを判定することとする

13

やってみよう

([data10.ipynb](#) と [email.csv](#) を Sc188 にダウンロードして実行)

- ワードクラウドの作成 (頻出単語の可視化)
 - python の wordcloud ライブラリと matplotlib ライブラリを使うと, 単語の出現頻度を文字の大きさに表した画像 (ワードクラウド) を作成・表示できる.
- スпамメールフィルターの作成 (ナイーブベイズによる判定)
 - python の scikit-learn (sklearn) というライブラリを使うと, ナイーブベイズを用いたテキストの分類器が作れる

15

ナイーブベイズによる迷惑メール判定

この $P(S|A_1 \cap \dots \cap A_k)$ と $P(\bar{S}|A_1 \cap \dots \cap A_k)$ の大小関係は, 計算式の右辺の分母が共通なので, $P(A_1|S) \dots P(A_k|S)P(S)$ と $P(A_1|\bar{S}) \dots P(A_k|\bar{S})P(\bar{S})$ の大小関係でもある. したがって, これらの値で比較すればよいことになる. この計算に用いる確率は, 過去の事例から簡単に計算できる

$$\begin{pmatrix} P(A_i|S) & \text{迷惑メールが語 } w_i \text{ を含む確率 } (i = 1 \dots n) \\ P(S) & \text{メールが迷惑メールである確率} \\ P(A_i|\bar{S}) & \text{非迷惑メールが語 } w_i \text{ を含む確率 } (i = 1 \dots n) \\ P(\bar{S}) & \text{メールが迷惑メールでない確率} \end{pmatrix}$$

14

やってみよう (時間があれば)

([Janome.ipynb](#) と [sangetsuki.txt](#) を Sc188 にダウンロードして実行)

- [Janome](#) による日本語形態素解析
 - janome ライブラリを用いると, 日本語形態素解析ができる
 - sangetsuki.txt は, 青空文庫から取得した中島敦の「山月記」を加工したもの
 - 日本語のワードクラウドも作成できる
- [以前](#)に紹介した [AIテキストマイニング](#) のページでは, 日本語テキストを対象にさまざまな文章データ処理を行えるので, 試してみよう

16